Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт

Про виконання лабораторної роботи №7

«ФУНКЦІЇ»

Виконав:
Студент групи ФЕП-13
Шмирко В.М.

Перевірив:
ас. Васюта В. М.

Львів 2026

Мета роботи:

Вивчити поняття і застосування функцій.

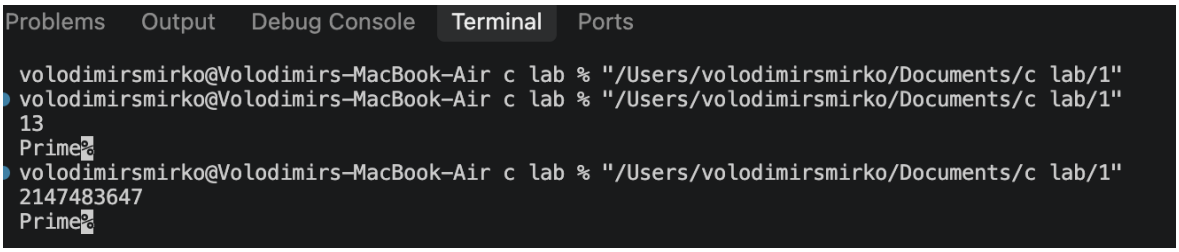
Хід роботи

1. Функція перевірки, чи введене число є простим (тест Міллера–Рабіна).

Реалізовано функцію mrtest(n) на основі тесту Міллера–Рабіна з базою a = 2. Спочатку відсікаються тривіальні випадки: n ≤ 1 — не просте, n = 2 або 3 — просте, парні та кратні 3 — не прості. Далі n−1 розкладається у вигляді 2^r · m (m непарне). Обчислюється x = 2^m mod n швидким піднесенням до степеня. Якщо x дорівнює 1 або n−1, число вважається простим. Інакше x кілька разів підноситься до квадрата (до r−1 разів); якщо на якомусь кроці з'являється n−1, тест пройдено, інакше число складене. У main() зчитується число і виводиться «Prime» або «non prime».

```c
#include <stdio.h>

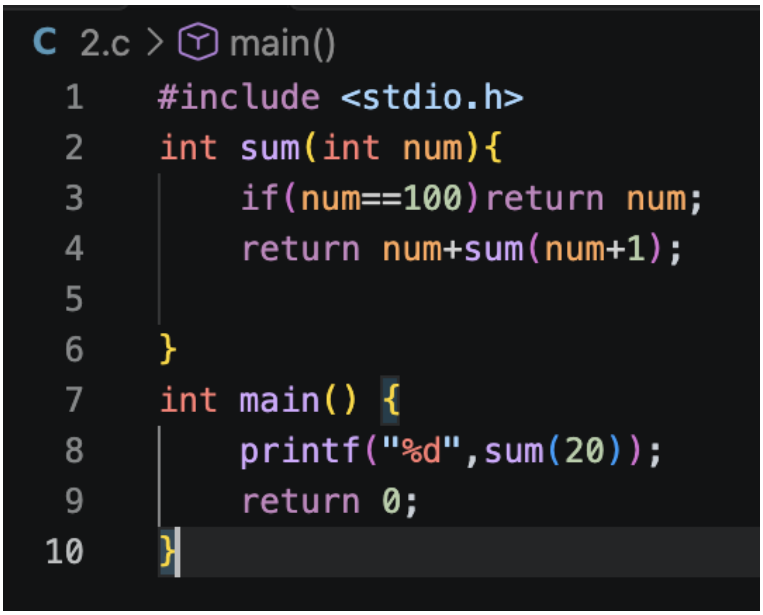
int number;
bool mrtest(int n){
    if(n<=1)return false;
    if(n<=3)return true;
    if(n%2==0||n%3==0)return false;
    //m
    int m = n-1,r=0;
    while(m%2 == 0){
        m/=2;
        r++;
    }
    //base
    //x=(a^m) mod n
    long long x=1,base=2;
    while (m>0) {      //x=r^m % n
    if (m%2==1) x = (x * base) % n;
    base = (base * base) % n;
    m /= 2;
    }
    //mod=1 or mod=-1(d0)
    if(x==1||x==n-1)return true;
    //d(r-1)
    for(int i=0;i<r-1;i++){
        x=(x*x)%n;
        if(x==n-1)return true;
    }
    return false;
}
int main() {
    scanf("%d",&number);
    if(mrtest(number)) printf("Prime");
    else printf("non prime");
    return 0;
}
```

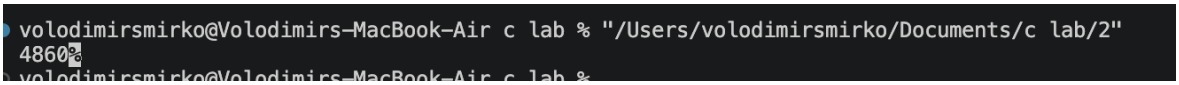


Приклади: 13 -> Prime; 2147483647 -> Prime.

2. Рекурсивне обчислення суми чисел від порядкового номера в журналі (20) до 100.

Функція sum(num) повертає num, якщо num == 100 (базовий випадок), інакше num + sum(num+1). Виклик sum(20) дає суму арифметичної прогресії 20 + 21 + … + 100.





Результат: sum(20) = 4860.

3. Розв’язок рівняння N·x + A = 0 методом половинного ділення (N = 20, A = 18).

Рівняння: 20x + 18 = 0. Функція f(x) = 20x + 18. Функція ans() реалізує бісекцію на інтервалі [−100; 100] з точністю eps = 0.001: на кожному кроці береться середина m, інтервал звужується за знаком добутку f(l)·f(m). Повертається (l+r)/2.

```c
#include <stdio.h>
double f(double x){
    return 20.0*x+18.0;
}
double ans(){
    double l= -100.0,r= 100.0,eps=0.001,m;
    while((r-l)>=eps){
        m=(l+r)/2.0;
        if( f(l)*f(m) <0)r=m;
        else l=m;
    }
    return (l+r)/2.0;
}
int main() {

    printf("%.4f",ans());
    return 0;
}
```

```
volodimirsmirko@Volodimirs-MacBook-Air c lab % "/Users/volodimirsmirko/Documents/c lab/3"
-0.8999%
```

Результат: x ≈ −0.8999 (точне значення −0.9).

Алгоритм завдання 1 (перевірка простоти)

1. Якщо n ≤ 1 — не просте; якщо n ≤ 3 — просте; якщо n парне або кратне 3 — не просте.
2. Записати n−1 = 2^r · m (непарне m).
3. Обчислити x = 2^m mod n; якщо x = 1 або x = n−1 — просте.
4. Інакше кілька разів піднести x до квадрата; якщо з’явиться n−1 — просте, інакше — ні.
5. У main вивести «Prime» / «non prime».

Алгоритм завдання 2 (рекурсивна сума)

1. Базовий випадок: якщо num == 100, повернути 100.
2. Інакше повернути num + sum(num + 1).
3. Викликати sum(20) і вивести результат.

Алгоритм завдання 3 (половинне ділення)

1. Задати f(x) = 20x + 18, інтервал [−100; 100], eps = 0.001.
2. Поки (r − l) ≥ eps: m = (l+r)/2; якщо f(l)·f(m) < 0 — r = m, інакше l = m.
3. Повернути (l+r)/2.

Висновок

Функції дозволяють розбити задачу на логічні частини з чітким входом і виходом через return. У перевірці простоти вся логіка тесту Міллера–Рабіна зосереджена в mrtest(), а main лише зчитує дані і друкує результат — код стає читабельнішим. Рекурсія для суми від 20 до 100 показує, як базовий випадок і рекурсивний виклик замінюють явний цикл; для невеликої глибини це зручно, хоча для великих діапазонів краще ітерація.

Метод половинного ділення для 20x + 18 = 0 демонструє, як функцію f(x) можна передавати в алгоритм пошуку кореня незалежно від конкретної формули. За заданою точністю eps = 0.001 отримано x ≈ −0.8999, що близьке до точного −0.9. Загалом функції в C — базовий засіб структурування програми, а рекурсія і чисельні методи добре лягають на цей механізм.